

Final Project: Blog Application (Express, MongoDB, Auth & Media Upload)

Overview

In this final project, you will build a full-featured Blog Application using:

- Express.js
- MongoDB
- Mongoose

This project simulates a real-world backend system with authentication, authorization, media uploads, and role-based access control.

Project Requirements

Build a Blog System including all the following features.

Core Features

Authentication & Authorization

- Register & Login
- Password hashing
- JWT Authentication
- Protected routes

Users CRUD

- Create, Update, Delete, Get All Users

Posts CRUD

- Create, Update, Delete, Get All Posts
- Return Global + Allowed Group Posts sorted by createdAt
- Get User Posts

Image Upload

- One or more images per post
- Upload using ImageKit
- Store image URLs

Groups

- Create groups
- Admins & Members
- Admins manage users & permissions
- Allowed users can post, others can only view

Post Ownership

- Users edit/delete only their own posts
- Super Admin can perform any action.

Middleware Requirements

- Authentication Middleware (JWT)
- Authorization Middleware (restrictTo: user/admin/super-admin)
- Multer + uploadOnImageKit

Error Handling

Global Error Middleware

```
class AppError extends Error {
  constructor(message, statusCode){
    super(message);
    this.statusCode=statusCode;
  }
}
```

Use it across the application.

Validation

Validate POST / PUT / PATCH requests using Joi or another validation library.

Database Design

User: username, email, password, role, timestamps

Post: title, content, images[], author, group, timestamps

Group: name, admins[], members[], permissions, timestamps

API Requirements

Auth: POST /auth/register, POST /auth/login

Users: CRUD

Posts: CRUD + Upload Images

Groups: Create, Add/Remove Users, Manage Permissions

Business Logic

- Only admins manage group users
- Only allowed users post in groups
- Only owner edits/deletes post
- Super admin overrides all rules

Deployment

Mandatory:

1. GitHub Repository
2. Vercel Deployment

Bonus

Comments, Likes, Pagination, Search Posts, Rate Limiting

Expected Project Structure

```
/project
■■■ controllers
■■■ models
■■■ routes
■■■ middleware
■■■ utils/AppError.js
■■■ config
■■■ server.js
```

■■■ app.js

Expected Outcome

Production-like backend, Authentication, Authorization, Media Upload, RBAC, Clean Architecture.

Notes

Follow clean code principles, MVC architecture, handle edge cases, and test using Postman.